WPT Day 6 Morning

Mon 26 Oct 26 · Regex-based form validation (password, phone, email), the Fetch API against JSONPlaceholder, async/await vs `.then` chains, populating an HTML table from fetched users, and three JSON object-literal variations — explained from first principles

---

Table of Contents

1. Why Manual Validation Breaks Down
2. Regular Expressions: One Formula for Many Rules
3. Password Regex Implementation
4. Phone and Email Validation
5. Why the Frontend Must Talk to a Server
6. The Fetch API
7. The Response Object and Promise Chains
8. Async/Await as the Preferred Style
9. HTTP Status Codes and What Fetch Actually Rejects
10. Populating a Table from Fetched Users
11. Dynamic Columns and Jagged Objects
12. JSON Object Variations
13. Combined Worked Example: Validated Form Plus Live Users Table
14. Quick Revision Checklist
15. Practice Exercises
16. Practice Answers

Why Manual Validation Breaks Down

Problem: required-field checks are not enough

Day 5 already covered the basic form-validation pattern: on submit (and often on `keyup`), read each field's value, check whether it is empty, show a live error message under the field, and keep an `isValid` flag so the form does not "succeed" while any field is wrong.

That pattern answers only one question: *did the user type anything?* Real-world forms ask harder questions:

- Is the password strong enough?
- Is the phone number a valid Indian mobile number?
- Does the email look like an address someone could actually use?

Without those rules, an empty-check form happily accepts `asd` as a password and `123` as a phone number. The server then either rejects the request or stores garbage. Client-side format rules exist so bad data never leaves the browser in the first place.

The naive approach: walk every character by hand

You *can* validate a password by hand. For each rule you invent a loop:

1. Walk the string and count length — reject if under 8.
2. Walk again and compare each character's ASCII code against **65–90** to detect an uppercase letter.
3. Walk again for lowercase, digits, and symbols.
4. Walk again to reject spaces.

Each rule is a separate iteration (or a pile of nested `if`s). The code is long, easy to get wrong, and painful to change when the product owner adds "must include at least one special character." The lecture's verdict is blunt: **this is not a practical approach for production code.**

> **Exam:** Manual character-by-character validation (ASCII ranges, separate loops per rule) works in theory but is verbose and time-consuming — regex replaces that pile of loops with one formula.

Regular Expressions: One Formula for Many Rules

Mechanism: one pattern, one `.test()` call

A **regular expression** (regex) is a compact formula that describes a family of strings. Instead of writing five loops, you write one pattern; JavaScript tests the user's string against that pattern in a **single call** and returns `true` or `false`.

In JavaScript a regex literal is written between **forward slashes**, not quotes:

```
const passwordRegex = /^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[\W_]).{8,}$/;
```

- Starts with `^` (beginning of the string) and ends with `$` (end of the string) so the whole value must match, not just a substring.
- Enclosed in `/…/`. If you wrap the same text in quotes — `"/^(?=.*[A-Z])…$/"` — you get a **plain string**, not a regex object, and `.test()` will not work the way you expect.

Background: `.test(string)` is a method on `RegExp` objects. It returns a boolean. Creating regex from first principles (lookaheads, character classes, quantifiers) is a deep topic of its own; **this session focuses only on how to obtain and apply a ready-made pattern**, not how to author every symbol by hand.

Where to get a regex

The lecture listed three practical sources:

Source	How you use it
regexlib.com	Browse / search a library of community patterns (session had connectivity issues live)
Web search	e.g. “regex for password”, “regex for Indian phone number”
GenAI (GPT, Gemini)	Describe the rules in plain English; receive a ready pattern

You describe requirements; you paste the result into a `const …Regex = /…/;` variable; you call `.test()`. Authoring regex deeply is deferred.

> **Pitfall:** Never put the regex in quotes. `"/abc/"` is a string. `/abc/` is a regular expression. Only the second has `.test()`.

Password Regex Implementation

The pattern used in class

```
const passwordRegex = /^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[\W_]).{8,}$/;
```

Rules this encodes (the lecture’s password policy):

- At least **8 characters**
- At least **1 uppercase** letter
- At least **1 lowercase** letter
- At least **1 number**
- At least **1 special character**
- **No spaces** (the pattern does not allow whitespace to satisfy the match)

Background (reading the pattern, optional): `(?=.*[A-Z])` is a *positive lookahead* — “somewhere ahead there must be an uppercase letter.” Similar lookaheads cover lowercase, digit (`\d`), and special (`[\W_]`). `.{8,}` means “eight or more of any character” for the actual match length. You do not need to derive this in an exam; you need to know that **one such formula replaces the manual loops**.

Two-step else-if validation

Empty and format are different failures. The lecture's logic keeps them separate:

```
let isValid = true;
const password = document.getElementById("password").value.trim();
const passwordError = document.getElementById("passwordError");

if (password.length === 0) {
    passwordError.innerHTML = "Password is required";
    isValid = false;
} else if (!passwordRegex.test(password)) {
    passwordError.innerHTML =
        "Required at least 8 characters, 1 uppercase, 1 lowercase, 1 number, 1 special
character";
    isValid = false;
} else {
    passwordError.innerHTML = "";
}
```

Mechanism, step by step:

1. If length is zero → show **"Password is required"** immediately; do **not** run the regex (an empty string failing the regex would produce a confusing format message).
2. Else if `passwordRegex.test(password)` is `false` → show the long **format** error.
3. Else → clear the error.
4. Set `isValid = false` in both error branches; it stays `true` only if neither branch fires.

Live demo observations

User action	What appears
Click Register with password empty	"Password is required"
Type a weak value such as `asd`	Format error (length / character-class rules fail)
Type a valid value (lecture demo cited `ASD@12345`; against the written regex include a lowercase too, e.g. `Asd@12345`)	Password error clears; remaining errors (username, phone, …) still show if those fields are wrong

Same check on `keyup`

The identical empty-vs-regex logic was also wired to the password field's `onkeyup` handler. Effect:

- Error appears **as the user types** an invalid value.

- Error **disappears as soon as** the regex is satisfied — no need to click Register to discover the password is finally OK.

> **Exam:** Validation order matters — check "required" first, then `.test(regex)`. Both branches set `isValid = false`. The same logic belongs on submit and on `keyup` for live feedback.

Phone and Email Validation

Same pattern, different formulas

Phone and email use the **identical** control-flow shape: required check → regex `.test()` → set `isValid` / show message. Only the pattern and the error text change.

Indian mobile number characteristics (from the live web search)

The lecture pulled an Indian mobile pattern with these properties:

1. Begins with `+91`
2. Followed by **exactly 10 digits**
3. The **first** of those 10 digits must be **6, 7, 8, or 9** (Indian mobile series)

Students were told to implement phone validation **independently** using the same `else if` + `.test()` structure already proven on password.

Email

Email regex can be sourced the same way (regexlib, search, or GenAI) and dropped into:

```
const emailRegex = /^…$/;  // paste a trusted email pattern

if (email.length === 0) {
    // "Email is required"
} else if (!emailRegex.test(email)) {
    // "Enter a valid email address"
} else {
    // clear error
}
```

Once every field follows this shape, **the entire form** is validated with one consistent approach — no special cases beyond the pattern string itself.

> **Exam:** Phone/email validation is not a new technique; it is the password technique with a different regex. Indian mobiles: `+91` + 10 digits starting with 6–9.

Why the Frontend Must Talk to a Server

Problem: a page alone cannot remember shared data

HTML, CSS, and client-side JavaScript can build a beautiful UI, but the browser tab is not the system of record. Login credentials, product catalogues, orders, and user lists live on a **server**. The frontend must:

- **Read** data (list users, search products)
- **Save** data (register, place order)
- **Update** and **delete** data

Without a network call, the UI can only reshuffle what it already has in memory.

Why the round-trip must be asynchronous

Two reasons, both from the lecture:

1. **Must not block other processes** in the browser — a frozen tab while waiting for the network is unacceptable.
2. **Must not trigger a full page reload** — modern apps update content **in place**.

Live analogy: Flipkart's search bar. As characters are typed, results appear underneath. The address bar does not flash; the whole page does not reload. That is fetch-and-respond happening asynchronously behind the scenes.

Background: Day 5 already established promises and `async` / `await` as the tools for waiting without blocking. Fetch is the browser's built-in API that returns those promises for HTTP.

The Fetch API

Mechanism

JavaScript's built-in `fetch(url)` starts an HTTP request:

Argument	Role
First: `url`	Target endpoint (string)
Second (optional): config object	e.g. `{ method: "POST" }` for non-GET verbs

- **GET** is the default — for reading data you usually pass only the URL.
- For **POST**, **PUT**, or **DELETE**, pass a second argument: `{ method: "POST" }` (and typically `headers` / `body` in real apps).

JSONPlaceholder as the demo server

Live demo URL:

```
https://jsonplaceholder.typicode.com/users
```

JSONPlaceholder is a **fake / dummy** backend. It returns a static array of JSON user objects. It mirrors real APIs: the frontend calls a URL; the backend returns structured JSON. No real database is required for learning the client side.

The critical mistake: treating `fetch` as synchronous

```
const response = fetch(url);  // WRONG expectation
```

`fetch()` returns a **Promise**, not the response data. Assigning the return value to `response` gives you a Promise object. You must **handle** that Promise before any user data is accessible.

> **Pitfall:** `const response = fetch(url)` does **not** give you the HTTP response body. It gives you a Promise that will eventually resolve to a Response object.

The Response Object and Promise Chains

What arrives when the Promise resolves

The resolved value is a full **Response** object, not the raw JSON. Useful properties mentioned in class:

Property	Meaning
`body`	Stream of the payload
`headers`	Response headers
`status`	HTTP status code (e.g. 200, 404)
`statusText`	Textual status
`type`	Response type
`url`	Final URL

`response.status` is how you read the numeric code. The actual data payload is **not** on those properties directly — you extract it with `response.json()`, which itself returns **another Promise** (parsing the body is also async).

Approach 1: `.then()` / `.catch()` (works, gets nested)

```
fetch("https://jsonplaceholder.typicode.com/users")
    .then(function (response) {
        return response.json();  // another Promise
    })
    .then(function (users) {
        console.log(users);      // actual array
    })
    .catch(function (err) {
        console.log("Network or parse failure", err);
    });
```

Mechanism:

1. First `.then()` receives the **Response** object.
2. Inside it you must call `response.json()` (or return it so the next `.then` receives the parsed data).
3. Because `.json()` returns another Promise, you need a **further** `.then()` (or a returned Promise that chains).
4. `.catch()` handles network failure / rejection.

The lecture's complaint: this leads to **nested callbacks** (`then` inside `then` thinking) even when you chain flatly — harder to scale mentally than sequential-looking `await` code.

Async/Await as the Preferred Style

Mechanism

Wrap the work in an **`async` function**. Use **`await`** to pause that function until each Promise settles. Use **`try` / `catch`** for failures.

```
async function fetchUsersData() {
    try {
        const response = await fetch(
            "https://jsonplaceholder.typicode.com/users"
        );
        const usersData = await response.json();
        console.log(usersData);
        // …use usersData to update the DOM
    } catch (err) {
        console.log("Request failed", err);
    }
}
```

Step by step:

1. `await fetch(url)` — pauses until the Response object arrives; assigns it directly to `response`.
2. `await response.json()` — pauses again; assigns the **actual data array** to `usersData`.
3. `catch` runs if the Promise rejects (network down, etc.).
4. The code stays **flat and readable** — preferred style in this course for scaling.

> **Exam:** Preferred path is `async` function + `await fetch` + `await response.json()` inside `try/catch`. `.then` chains work but nest and are harder to maintain.

HTTP Status Codes and What Fetch Actually Rejects

Success vs failure series (lecture clarification)

Series	Rough meaning	Promise behaviour (as taught)
1xx, 2xx, 3xx	Treated as success path	Promise resolves; `.then` / `try` runs
4xx, 5xx	Treated as failure path in the mental model	Application should treat as failure

Important nuance the lecture called out

A **404** response still arrives as a **Response object**. With the Fetch API, HTTP error status codes do **not** automatically reject the Promise the way a network outage does. The Promise often **resolves** with `response.status === 404`; you must **check** `status` **manually** if you need to branch on Not Found.

Background: In other words: "no internet / DNS failure" → rejection → `catch`. "Server answered with 404" → still a Response → inspect `response.ok` or `response.status` yourself. The lecture's series table is the exam-facing simplification; the 404 nuance is the pitfall that catches students who assume every bad status jumps to `catch`.

> **Pitfall:** Do not assume `catch` runs for every 404. Fetch gives you the Response; check `status` when the UI must treat Not Found as an error.

Populating a Table from Fetched Users

Goal

Display the fetched users array in an HTML table, filled **dynamically on button click**, with **no page reload**.

HTML skeleton

```
<button onclick="fetchUsersData()">Fetch Users Data</button>

<table border="1">
    <thead>
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Username</th>
            <th>Email</th>
            <th>Phone</th>
        </tr>
    </thead>
    <tbody id="tbody">
        <!— empty on page load; filled by JavaScript —>
    </tbody>
</table>
```

- `<thead>` holds a **fixed** header row.

- `<tbody id="tbody">` starts **empty**.
- Button calls `fetchUsersData()`.

Per-user DOM construction (intentionally verbose)

Inside the async function, after `usersData` is ready:

```
const tbody = document.getElementById("tbody");
tbody.innerHTML = "";  // optional clear before refill

for (let i = 0; i < usersData.length; i++) {
    const row = document.createElement("tr");

    const idCol = document.createElement("td");
    const nameCol = document.createElement("td");
    const usernameCol = document.createElement("td");
    const emailCol = document.createElement("td");
    const phoneCol = document.createElement("td");

    const { id, name, username, email, phone } = usersData[i];

    idCol.innerHTML = id;
    nameCol.innerHTML = name;
    usernameCol.innerHTML = username;
    emailCol.innerHTML = email;
    phoneCol.innerHTML = phone;

    row.appendChild(idCol);
    row.appendChild(nameCol);
    row.appendChild(usernameCol);
    row.appendChild(emailCol);
    row.appendChild(phoneCol);

    tbody.appendChild(row);
}
```

Mechanism per iteration:

1. `document.createElement("tr")` — new row.
2. Five `document.createElement("td")` calls — one column each.
3. **Destructure** the current object: `const { id, name, username, email, phone } = usersData[i]`.
4. Assign each value with `.innerHTML`.
5. `row.appendChild(…)` for all five cells; then `tbody.appendChild(row)`.

Live demo behaviour

- Click the button → table fills with users; **no reload**.
- Reload the page → table is empty again (DOM state is not persisted).
- Click again → table refills.

Why so many lines?

The instructor noted this implementation is **intentionally verbose** to expose the underlying mechanics. **React and Angular** will collapse this into far less boilerplate later — the pain here is deliberate context so those frameworks feel earned, not magical.

> **Exam:** Fetch → `response.json()` → loop → `createElement` for `tr` / `td` → destructure fields → `appendChild` to `tbody`. Header stays in HTML; body is built in JS.

Dynamic Columns and Jagged Objects

Student question: can column count be dynamic?

Yes — but only if every object in the array has the same keys in the same order.

If objects have **jagged** key sets (object A has `phone`, object B does not; keys appear in different orders), a naive "loop all keys of each object" approach **breaks** alignment — columns would not mean the same thing across rows.

Safe strategy when keys are uniform

1. Read keys from the **first** object (`Object.keys(usersData[0])`).
2. Loop those keys to build header cells and, for each row, data cells in the same order.

If key sets differ, normalise the data on the server (or pick a fixed column list in the client) before rendering.

JSON Object Variations

Three practical variations were demonstrated in a live `json-demo.html` file.

1. Variable as a value (standard)

```
const productPrice = 500;
const obj = { price: productPrice };
// → { price: 500 }
```

- Key name is the **literal** `price`.
- Value is whatever `productPrice` holds at runtime.

2. Variable name as shorthand key

```
const pid = 1;
const pName = "Pen";
const productPrice = 20;
const qty = 5;

const product = { pid, pName, productPrice, qty };
// equivalent to:
// { pid: pid, pName: pName, productPrice: productPrice, qty: qty }
```

When the **variable name and desired key name are the same**, write the name once. Useful for packing several locals into one object without repetition.

3. Variable value as a dynamic key (computed property)

```
const xyz = "price";

const a = { xyz: 1000 };
// → { xyz: 1000 }   // literal key "xyz"

const b = { [xyz]: 1000 };
// → { price: 1000 } // key is the *value* of xyz
```

Without square brackets, `{ xyz: 1000 }` always creates a key literally named `"xyz"`.
With square brackets, `{ [xyz]: 1000 }` uses the **runtime value** of `xyz` as the key. Essential when key names are decided at runtime.

Bracket notation for reading

```
product.pid;       // dot notation
product["pid"];    // bracket notation — equivalent for fixed keys

const key = "pid";
product[key];      // dynamic read — only brackets work here
```

`product.pid` and `product["pid"]` are equivalent for a fixed key. Bracket notation also allows `product[someVariable]` when the key itself is in a variable.

Variation	Syntax	Resulting key
Value from variable	`{ price: productPrice }`	always `"price"`
Shorthand	`{ productPrice }`	`"productPrice"` (name of variable)
Computed key	`{ [xyz]: 1000 }`	whatever string `xyz` currently holds
Literal trap	`{ xyz: 1000 }`	always `"xyz"`, ignores variable value

> **Exam:** `{ productPrice }` equals `{ productPrice: productPrice }`. `{ [xyz]: 1000 }` uses the variable's value as the key; `{ xyz: 1000 }` does not.

Combined Worked Example: Validated Form Plus Live Users Table

One page that ties regex validation, fetch, async/await, and dynamic table rows together. Open as a local HTML file (needs network access for JSONPlaceholder).

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8" />
    <title>WPT Day 6 Morning – Combined Demo</title>
    <style>
        body { font-family: Georgia, serif; max-width: 720px; margin: 2rem auto; }
        label { display: block; margin-top: 0.75rem; }
        input { width: 100%; padding: 0.4rem; box-sizing: border-box; }
        .err { color: #b00020; font-size: 0.9rem; min-height: 1.2em; }
        .ok { border: 2px solid #2e7d32; }
        .bad { border: 2px solid #b00020; }
        table { border-collapse: collapse; width: 100%; margin-top: 1.5rem; }
        th, td { border: 1px solid #333; padding: 0.4rem 0.6rem; text-align: left; }
        button { margin-top: 1rem; margin-right: 0.5rem; padding: 0.45rem 0.9rem; }
    </style>
</head>
<body>
    <h1>Register + Fetch Users</h1>

    <form id="regForm" onsubmit="return validateForm(event)">
        <label for="username">Username</label>
        <input id="username" type="text" onkeyup="validateUsernameLive()" />
```

```
        <div id="usernameError" class="err"></div>

        <label for="password">Password</label>
        <input id="password" type="password" onkeyup="validatePasswordLive()" />
        <div id="passwordError" class="err"></div>

        <label for="phone">Phone (+91…)</label>
        <input id="phone" type="text" onkeyup="validatePhoneLive()" />
        <div id="phoneError" class="err"></div>

        <label for="email">Email</label>
        <input id="email" type="text" onkeyup="validateEmailLive()" />
        <div id="emailError" class="err"></div>

        <button type="submit">Register</button>
    </form>

    <button type="button" onclick="fetchUsersData()">Fetch Users Data</button>

    <table>
        <thead>
            <tr>
                <th>ID</th>
                <th>Name</th>
                <th>Username</th>
                <th>Email</th>
                <th>Phone</th>
            </tr>
        </thead>
        <tbody id="tbody"></tbody>
    </table>

    <script>
        const passwordRegex =
            /^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[\W_]).{8,}$/;
        // Indian mobile: +91 then 10 digits starting with 6-9
        const phoneRegex = /^\+91[6-9]\d{9}$/;
        const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

        function setFieldState(inputId, errorId, message, ok) {
            const input = document.getElementById(inputId);
            const err = document.getElementById(errorId);
            err.innerHTML = message;
            input.classList.remove("ok", "bad");
            if (message === "" && document.getElementById(inputId).value.trim() !==
"") {
                input.classList.add("ok");
            } else if (message !== "") {
                input.classList.add("bad");
            }
            return ok;
```

```
        }

        function checkUsername() {
            const v = document.getElementById("username").value.trim();
            if (v.length === 0) {
                return setFieldState("username", "usernameError", "Username is
required", false);
            }
            return setFieldState("username", "usernameError", "", true);
        }

        function checkPassword() {
            const v = document.getElementById("password").value.trim();
            if (v.length === 0) {
                return setFieldState("password", "passwordError", "Password is
required", false);
            }
            if (!passwordRegex.test(v)) {
                return setFieldState(
                    "password",
                    "passwordError",
                    "Required at least 8 characters, 1 uppercase, 1 lowercase, 1
number, 1 special character",
                    false
                );
            }
            return setFieldState("password", "passwordError", "", true);
        }

        function checkPhone() {
            const v = document.getElementById("phone").value.trim();
            if (v.length === 0) {
                return setFieldState("phone", "phoneError", "Phone is required",
false);
            }
            if (!phoneRegex.test(v)) {
                return setFieldState(
                    "phone",
                    "phoneError",
                    "Use +91 followed by 10 digits starting with 6-9",
                    false
                );
            }
            return setFieldState("phone", "phoneError", "", true);
        }

        function checkEmail() {
            const v = document.getElementById("email").value.trim();
            if (v.length === 0) {
                return setFieldState("email", "emailError", "Email is required",
false);
```

```
            }
            if (!emailRegex.test(v)) {
                return setFieldState("email", "emailError", "Enter a valid email
address", false);
            }
            return setFieldState("email", "emailError", "", true);
        }

        function validateUsernameLive() { checkUsername(); }
        function validatePasswordLive() { checkPassword(); }
        function validatePhoneLive() { checkPhone(); }
        function validateEmailLive() { checkEmail(); }

        function validateForm(event) {
            event.preventDefault();
            let isValid = true;
            if (!checkUsername()) isValid = false;
            if (!checkPassword()) isValid = false;
            if (!checkPhone()) isValid = false;
            if (!checkEmail()) isValid = false;
            if (isValid) {
                alert("Form OK - ready to send to server");
            }
            return false;
        }

        async function fetchUsersData() {
            try {
                const response = await fetch(
                    "https://jsonplaceholder.typicode.com/users"
                );
                if (!response.ok) {
                    console.log("HTTP status", response.status);
                    return;
                }
                const usersData = await response.json();
                const tbody = document.getElementById("tbody");
                tbody.innerHTML = "";

                for (let i = 0; i < usersData.length; i++) {
                    const row = document.createElement("tr");
                    const idCol = document.createElement("td");
                    const nameCol = document.createElement("td");
                    const usernameCol = document.createElement("td");
                    const emailCol = document.createElement("td");
                    const phoneCol = document.createElement("td");

                    const { id, name, username, email, phone } = usersData[i];
                    idCol.innerHTML = id;
                    nameCol.innerHTML = name;
                    usernameCol.innerHTML = username;
```

```
                    emailCol.innerHTML = email;
                    phoneCol.innerHTML = phone;

                    row.appendChild(idCol);
                    row.appendChild(nameCol);
                    row.appendChild(usernameCol);
                    row.appendChild(emailCol);
                    row.appendChild(phoneCol);
                    tbody.appendChild(row);
                }
            } catch (err) {
                console.log("Fetch failed", err);
            }
        }
    </script>
</body>
</html>
```

End-to-end trace

1. User types in Password → each `keyup` runs `checkPassword`: empty → "Password is required"; `asd` → format message; `Asd@12345` → error clears, green border.
2. Phone must look like `+918765432109` (example shape): `+91` + digit 6–9 + nine more digits.
3. Register runs all four checks; `isValid` stays true only if every check returns true.
4. Fetch Users Data → `await fetch` → `await response.json()` → classic `for` loop builds rows with destructuring — Flipkart-style update **without reload**.
5. Reload clears `tbody`; fetch again rebuilds it.

This matches the homework direction from the lecture: reuse the CSS-styled form, add real-time validation on `keyup`, red/green borders, and descriptive per-field errors — then (as practised here) combine with fetch-driven UI updates.

Quick Revision Checklist

Topic	One-line takeaway
Manual validation	Character loops / ASCII ranges work but are impractical for production rules
Regex literal	Written in `/…/` with `^` … `$`; quotes make a string, not a RegExp
Password check	Required first, then `passwordRegex.test`; same logic on submit and `keyup`
Phone / email	Same else-if + `.test()` shape; Indian mobile: `+91` + 10 digits starting 6–9

Topic	One-line takeaway
Why fetch	Frontend must read/write server data without blocking or full reload
`fetch(url)`	Returns a Promise; GET needs only the URL; other methods need `{ method }`
Response	Object with `status`, etc.; payload via `response.json()` (another Promise)
async/await	Preferred flat style: `await fetch`, `await response.json()`, `try/catch`
Status pitfall	404 still yields a Response — check `status` / `ok` yourself
Dynamic table	Loop users → `createElement` tr/td → destructure → `appendChild` to `tbody`
Jagged keys	Dynamic columns only if every object shares the same keys/order
JSON shorthand	`{ productPrice }` ≡ `{ productPrice: productPrice }`
Computed key	`{ [xyz]: 1000 }` uses variable value as key; `{ xyz: 1000 }` does not

Practice Exercises

Concept

1. Why is checking only `password.length === 0` insufficient for a registration form?
2. What goes wrong if you write `const passwordRegex = "/^(?=.*[A-Z])…$/";` (with quotes)?
3. Why must the fetch-and-respond cycle be asynchronous? Give both lecture reasons.
4. After `const response = await fetch(url)`, do you already have the users array? What else must you call?
5. Explain the difference between `{ xyz: 1000 }` and `{ [xyz]: 1000 }` when `xyz = "price"`.

Application

6. Write the else-if skeleton (no HTML) that validates `phone` with `phoneRegex`, setting `isValid` and an error string variable.
7. Write an `async function loadUsers()` that fetches `https://jsonplaceholder.typicode.com/users`, parses JSON, and `console.log`s the array, with `try/catch`.
8. Given `const pid = 10; const pName = "Book";`, build a product object using **shorthand** property names only.

Predict output

9. `passwordRegex.test("ASD@12345")` with the lecture password regex — `true` or `false`? Why?
10. `passwordRegex.test("asd")` — `true` or `false`? Why?
11. If `fetch` receives HTTP 404, does the Promise always reject into `catch`? What should you check?

Spot the bug

12. Find and fix the bugs:

```
async function fetchUsersData() {
    const response = fetch("https://jsonplaceholder.typicode.com/users");
    const usersData = response.json();
    for (let i = 0; i < usersData.length; i++) {
        console.log(usersData[i].name);
    }
}
```

13. Find the logic bug:

```
if (!passwordRegex.test(password)) {
    passwordError.innerHTML =
        "Required at least 8 characters, 1 uppercase, 1 lowercase, 1 number, 1 special
character";
    isValid = false;
} else if (password.length == 0) {
    passwordError.innerHTML = "Password is required";
    isValid = false;
}
```

Practice Answers

1. Empty-check only proves something was typed, not that it meets strength/format rules. Weak values like `asd` would pass and reach the server as bad data.

2. Quotes create a **string**, not a `RegExp`. You lose proper `.test()` behaviour (or must convert with `new RegExp` carefully). Use `/^(?=.*[A-Z])…$/` with slashes.

3. (a) Must not block other browser work while waiting on the network. (b) Must not force a full page reload — update content in place (Flipkart search example).

4. No — `response` is the Response object. Call `await response.json()` to get the users array.

5. `{ xyz: 1000 }` → `{ xyz: 1000 }` (literal key). `{ [xyz]: 1000 }` → `{ price: 1000 }` (computed key from the variable's value).

6.

```
let isValid = true;
let phoneError = "";
const phone = /* value from input */;

if (phone.length === 0) {
    phoneError = "Phone is required";
    isValid = false;
} else if (!phoneRegex.test(phone)) {
    phoneError = "Invalid Indian mobile number";
    isValid = false;
} else {
    phoneError = "";
}
```

7.

```
async function loadUsers() {
    try {
        const response = await fetch(
            "https://jsonplaceholder.typicode.com/users"
        );
        const users = await response.json();
        console.log(users);
    } catch (err) {
        console.log(err);
    }
}
```

8.

```
const product = { pid, pName };
```

9. Against the exact lecture pattern `/^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[\W_]).{8,}$/`, `ASD@12345` → `false` (no lowercase). Use `Asd@12345` → `true` (has upper, lower, digit, special, length ≥ 8). The live session cited `ASD@12345` as clearing the error — treat that as a demo moment; the written regex still demands a lowercase letter.

10. `false` — too short; missing uppercase, digit, and special character.

11. Not always. Fetch often **resolves** with a Response whose `status` is 404. Inspect `response.status` or `response.ok`; do not rely on `catch` alone for HTTP error codes.

12. Missing `await` on both `fetch` and `.json()`. Fixed:

```
async function fetchUsersData() {
    const response = await fetch(
        "https://jsonplaceholder.typicode.com/users"
    );
    const usersData = await response.json();
    for (let i = 0; i < usersData.length; i++) {
        console.log(usersData[i].name);
    }
}
```

13. Order is reversed. An empty password fails the regex first and shows the **format** message instead of "Password is required." Check `password.length === 0` **before** `.test()`.

Next steps from lecture: finish the signup form with regex validation on the existing CSS-styled form — real-time `keyup` validation, red border when invalid, green when valid, and a clear error message per field.